

Secure Login System

Project Overview

Develop a secure login web application using React.js, Node.js, Express.js, and MongoDB.

Key Features

- User registration and login using hashed passwords (bcrypt)
 - Input validation and protection from injection attacks
 - Session/JWT authentication with logout feature
 - Protected dashboard page
 - Optional Two-Factor Authentication (2FA)
 - Responsive frontend design
-

Technologies Used

Frontend

- React.js
- HTML
- CSS
- Axios
- React Router DOM

Backend

- Node.js
 - Express.js
 - MongoDB
 - Mongoose
 - bcryptjs
 - jsonwebtoken
 - dotenv
 - express-validator
 - cors
-

Folder Structure

```
secure-login-system/  
|  
├─ client/  
|   └─ src/  
|       ├── pages/  
|       ├── components/  
|       ├── App.js  
|       └─ index.js  
|  
├─ server/  
|   ├── config/  
|   ├── controllers/  
|   ├── middleware/  
|   ├── models/  
|   ├── routes/  
|   ├── server.js  
|   └─ .env
```

Backend Setup

Step 1: Initialize Backend

```
mkdir server  
cd server  
npm init -y
```

Step 2: Install Packages

```
npm install express mongoose bcryptjs jsonwebtoken dotenv cors express-validator  
npm install nodemon --save-dev
```

server.js

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const dotenv = require('dotenv');

dotenv.config();

const app = express();

app.use(express.json());
app.use(cors());

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB Connected'))
  .catch(err => console.log(err));

app.use('/api/auth', require('./routes/authRoutes'));

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

.env File

```
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=secure_secret_key
```

User Model

models/User.js

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
```

```
username: {
  type: String,
  required: true,
},
email: {
  type: String,
  required: true,
  unique: true,
},
password: {
  type: String,
  required: true,
},
});

module.exports = mongoose.model('User', UserSchema);
```

Authentication Routes

routes/authRoutes.js

```
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const { body, validationResult } = require('express-validator');
const User = require('../models/User');

const router = express.Router();

// Register
router.post(
  '/register',
  [
    body('username').notEmpty(),
    body('email').isEmail(),
    body('password').isLength({ min: 6 }),
  ],
  async (req, res) => {
    const errors = validationResult(req);

    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }
  }
```

```

const { username, email, password } = req.body;

try {
  let user = await User.findOne({ email });

  if (user) {
    return res.status(400).json({ msg: 'User already exists' });
  }

  const salt = await bcrypt.genSalt(10);
  const hashedPassword = await bcrypt.hash(password, salt);

  user = new User({
    username,
    email,
    password: hashedPassword,
  });

  await user.save();

  res.json({ msg: 'User registered successfully' });
} catch (err) {
  res.status(500).send('Server Error');
}
};

// Login
router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  try {
    const user = await User.findOne({ email });

    if (!user) {
      return res.status(400).json({ msg: 'Invalid credentials' });
    }

    const isMatch = await bcrypt.compare(password, user.password);

    if (!isMatch) {
      return res.status(400).json({ msg: 'Invalid credentials' });
    }

    const payload = {
      user: {
        id: user.id,
      },

```

```

    };

    jwt.sign(
      payload,
      process.env.JWT_SECRET,
      { expiresIn: '1h' },
      (err, token) => {
        if (err) throw err;

        res.json({ token });
      }
    );
  } catch (err) {
    res.status(500).send('Server Error');
  }
});

module.exports = router;

```

Frontend Setup

Step 1: Create React App

```

npx create-react-app client
cd client
npm install axios react-router-dom

```

App.js

```

import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Register from './pages/Register';
import Login from './pages/Login';
import Dashboard from './pages/Dashboard';

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Login />} />
        <Route path="/register" element={<Register />} />

```

```

        <Route path='/dashboard' element={<Dashboard />} />
      </Routes>
    </Router>
  );
}

export default App;

```

Register Page

pages/Register.js

```

import React, { useState } from 'react';
import axios from 'axios';

function Register() {
  const [formData, setFormData] = useState({
    username: '',
    email: '',
    password: '',
  });

  const { username, email, password } = formData;

  const onChange = (e) =>
    setFormData({ ...formData, [e.target.name]: e.target.value });

  const onSubmit = async (e) => {
    e.preventDefault();

    try {
      const res = await axios.post(
        'http://localhost:5000/api/auth/register',
        formData
      );

      alert(res.data.msg);
    } catch (err) {
      console.log(err.response.data);
    }
  };

  return (
    <div>

```

```

    <h2>Register</h2>

    <form onSubmit={onSubmit}>
      <input
        type='text'
        name='username'
        placeholder='Username'
        value={username}
        onChange={onChange}
      />

      <input
        type='email'
        name='email'
        placeholder='Email'
        value={email}
        onChange={onChange}
      />

      <input
        type='password'
        name='password'
        placeholder='Password'
        value={password}
        onChange={onChange}
      />

      <button type='submit'>Register</button>
    </form>
  </div>
);
}

export default Register;

```

Login Page

pages/Login.js

```

import React, { useState } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-dom';

function Login() {

```



```

const navigate = useNavigate();

const [formData, setFormData] = useState({
  email: '',
  password: '',
});

const { email, password } = formData;

const onChange = (e) =>
  setFormData({ ...formData, [e.target.name]: e.target.value });

const onSubmit = async (e) => {
  e.preventDefault();

  try {
    const res = await axios.post(
      'http://localhost:5000/api/auth/login',
      formData
    );

    localStorage.setItem('token', res.data.token);

    navigate('/dashboard');
  } catch (err) {
    console.log(err.response.data);
  }
};

return (
  <div>
    <h2>Login</h2>

    <form onSubmit={onSubmit}>
      <input
        type='email'
        name='email'
        placeholder='Email'
        value={email}
        onChange={onChange}
      />

      <input
        type='password'
        name='password'
        placeholder='Password'
        value={password}
        onChange={onChange}

```

```
        />

        <button type='submit'>Login</button>
      </form>
    </div>
  );
}

export default Login;
```

Dashboard Page

pages/Dashboard.js

```
import React from 'react';

function Dashboard() {
  const logout = () => {
    localStorage.removeItem('token');
    window.location.href = '/';
  };

  return (
    <div>
      <h1>Welcome to Dashboard</h1>
      <button onClick={logout}>Logout</button>
    </div>
  );
}

export default Dashboard;
```

Security Features Implemented

Password Hashing

Passwords are securely stored using bcrypt hashing.

Input Validation

All user input is validated using express-validator.

JWT Authentication

Secure tokens are generated after successful login.

Injection Protection

Mongoose prevents common injection attacks.

Logout Feature

Users can securely logout by clearing tokens.

Optional 2FA

Google Authenticator or OTP-based verification can be added.

Running the Project

Start Backend

```
cd server  
npm run dev
```

Start Frontend

```
cd client  
npm start
```

Expected Output

- Users can register securely
- Passwords are encrypted
- Users can login/logout
- Protected dashboard access

- Improved security against attacks
-

Conclusion

The Secure Login System project demonstrates secure authentication using modern web technologies. Password hashing, token authentication, validation, and session handling improve security and reduce unauthorized access risks.